

MCP 工具集成框架

面试准备文档

基于 JSON-RPC 2.0 的标准化工具集成协议 | GitHub: tanghui2/mcp-tool-framework

项目类型	Agent 大模型工程师面试项目
技术栈	Python + Pydantic V2 + aiohttp + asyncio
核心协议	MCP (Model Context Protocol) / JSON-RPC 2.0
GitHub 仓库	github.com/tanghui2/mcp-tool-framework
文档版本	v1.0

一、项目使用场景

1.1 项目定位

MCP 工具集成框架是一个基于 MCP (Model Context Protocol) 协议的标准化工具集成平台，通过 JSON-RPC 2.0 消息格式实现工具的注册、发现、调用和远程通信。框架提供了完整的协议层（消息序列化、传输层、服务器）、工具层（基类、注册中心、内置工具）和客户端层，支持进程内直连、SSE、WebSocket 三种传输方式，解决大语言模型 Agent 工具集成中的「工具碎片化」和「接口不统一」问题。

1.2 典型使用场景

- Agent 工具扩展：为 LLM Agent 提供标准化的工具调用接口，Agent 通过 MCP 客户端连接到工具服务器，动态发现可用工具并按需调用，无需硬编码工具列表。
- 微服务工具网关：在企业内部部署 MCP 服务器作为工具网关，统一管理搜索、数据库查询、文件操作等工具，多个 Agent 共享同一工具池，避免重复开发。
- 工具市场与发现：通过 tools/list 方法动态发现服务器上的工具，支持按分类过滤，实现「工具即服务」的按需加载模式。
- 远程工具注册：通过 tools/register 方法远程注册工具元信息，支持运行时动态扩展工具集，适用于插件化架构场景。
- 测试与调试：InProcessTransport 提供进程内直连模式，无需网络通信即可完成完整的协议链路测试，适合 CI/CD 环境和单元测试。
- 多传输协议适配：SSE 适合服务器推送场景（如实时数据流），WebSocket 适合双向频繁交互场景，InProcess 适合单进程高性能场景，框架统一了三者的 API。

1.3 适用人群与价值

该框架适用于需要将多种工具能力标准化集成到 LLM Agent 中的场景，为开发者提供了从协议层到应用层的完整工程化范式：包括 MCP 协议实现、传输层抽象、工具生命周期管理（注册/注销/状态监控/调用统计）、异步消息分发、请求-响应匹配等。对于企业而言，可作为内部工具治理的标准化方案；对于开发者而言，是理解 MCP 协议原理、异步通信架构和工具集成设计的最佳实践样本。

二、项目整体框架

2.1 分层架构

项目采用分层架构设计，自下而上分为：基础设施层、协议层、工具层、客户端层、应用层。

层级	模块	职责
----	----	----

基础设施层	config.py / logger.py / models.py	单例配置管理、日志、Pydantic 数据模型
协议层	protocol/message.py	JSON-RPC 2.0 消息序列化/反序列化、标准方法与错误码
协议层	protocol/transport.py	InProcess / SSE / WebSocket 三种传输层实现
协议层	protocol/server.py	MCP 服务器：消息分发、方法处理器、生命周期管理
工具层	tools/base.py	ToolBase 抽象基类：参数校验、安全执行、Schema 暴露
工具层	tools/registry.py	ToolRegistry 注册中心：注册/注销/查询/执行/统计
工具层	tools/builtin.py	6 个内置工具：Echo / Calculator / TextProcessor 等
客户端层	client/_init_.py	MCPClient：请求-响应匹配、高层 API、通知处理
应用层	run.py	演示脚本：进程内通信与远程注册两个场景

2.2 MCP 协议设计（核心）

MCP 协议基于 JSON-RPC 2.0 标准，定义了四种消息类型：请求（Request）、响应（Response）、通知（Notification）和错误（Error）。MessageSerializer 负责消息的序列化与反序列化，MCPMethods 定义了标准方法名，MCPErrorCodes 定义了标准错误码。

消息格式示例：

```
{
  "jsonrpc": "2.0",
  "id": "uuid",
  "method": "tools/call",
  "params": {
    "name": "calculator",
    "arguments": {
      "expression": "1+2"
    }
  }
}
```

方法	描述	方向
tools/list	列出所有已注册工具	客户端 -> 服务器
tools/call	调用指定工具	客户端 -> 服务器
tools/register	远程注册工具（仅元信息）	客户端 -> 服务器
tools/unregister	注销工具	客户端 -> 服务器
server/info	获取服务器信息	客户端 -> 服务器
server/ping	心跳检测	客户端 -> 服务器
server/shutdown	关闭服务器	客户端 -> 服务器

2.3 传输层设计（三种传输方式）

传输层通过 TransportBase 抽象基类统一接口（start/stop/send/receive/is_connected），支持三种具体实现，适用于不同场景：

- InProcessTransport (进程内传输)：使用 `asyncio.Queue` 实现消息直连，无需网络通信，适用于测试和单进程场景。设计为「传输对」工厂，通过 `server_side()` / `client_side()` 返回真正的 `TransportBase` 实现。
- SSETransport (Server-Sent Events)：基于 `aiohttp` 实现，服务器通过 SSE 流推送消息，客户端通过 HTTP POST 发送请求。适用于服务器主动推送场景。
- WebSocketTransport：基于 `aiohttp` WebSocket 实现双向通信，适用于频繁交互的实时场景。服务器和客户端通过同一个 WebSocket 连接收发消息。

2.4 工具管理机制 (ToolRegistry)

ToolRegistry 是工具的注册中心，管理工具的完整生命周期：

- 注册：`register(tool)` 注册 `ToolBase` 实例，`register_tool_dict(data)` 通过字典远程注册元信息。注册时创建 `ToolInfo` 记录（含 `tool_id`、状态、调用统计），本地工具状态为 `ACTIVE`，远程工具为 `INACTIVE`。
- 查询：`list_tools(category)` 按分类过滤，`list_names()` 返回所有工具名，`get_tool(name)` 获取实例。
- 执行：`execute_tool(name, arguments)` 异步执行工具，内部调用 `safe_execute` 包装异常，通过 `asyncio.wait_for` 实现超时控制，自动更新调用次数和错误次数统计。
- 状态管理：工具状态分为 `ACTIVE` / `INACTIVE` / `ERROR` 三种。业务错误 (`ToolError`) 不改变状态，工具仍可用；只有严重内部错误才标记为 `ERROR`。
- 注销：`unregister(name)` 同时移除工具实例和元信息。
- 统计：`get_stats()` 返回总工具数、活跃数、调用次数、错误次数、整体成功率。

2.5 工具基类设计 (ToolBase)

ToolBase 是所有工具的抽象基类，采用模板方法模式：

- Schema 暴露：通过 `get_schema()` 方法返回 `ToolSchema`（名称、描述、分类、参数 Schema、返回值 Schema），供 `tools/list` 接口返回给客户端。
- 参数校验：`validate_params()` 基于 JSON Schema 校验必填参数和类型，支持 `string/integer/number/boolean/array/object` 类型检查，注意处理 `bool` 是 `int` 子类的边界情况。
- 安全执行：`safe_execute()` 包装 `execute()`，捕获所有异常返回结构化结果（`success/output/error/execution_time`），避免异常冒泡到消息循环。区分 `ToolError`（业务错误）和未知异常（内部错误）。
- 抽象方法：`execute(params)` 是子类必须实现的异步方法，接收参数字典返回任意 JSON 可序列化结果。

2.6 客户端设计 (MCPClient)

MCPClient 是与 MCP 服务器通信的客户端，核心是通过 request_id 匹配请求和响应：

- 请求-响应匹配：每条请求生成唯一 UUID 作为 id，存入 _pending 字典 (id -> Future)。后台 _read_loop 读取响应消息，根据 id 匹配 Future 并 set_result / set_exception。
- 超时控制：asyncio.wait_for 设置 request_timeout (默认 30s)，超时自动取消并从 _pending 移除。
- 通知处理：通知消息 (无 id) 通过 on_notification(method, handler) 注册的处理器异步处理。
- 高层 API：ping()、server_info()、list_tools()、call_tool()、register_tool()、unregister_tool()，封装了 _request / _notify 底层方法。
- 生命周期：start() 启动传输层和读取循环，stop() 取消所有 pending Future 并关闭传输层。

2.7 核心执行流程

- 步骤 1 初始化：创建 InProcessTransport，获取 server_side 和 client_side 两个传输句柄。创建 ToolRegistry 并注册内置工具。
- 步骤 2 启动服务器：MCPServer.set_tool_registry(registry) 注入工具注册中心，server.start() 启动传输层并创建 _dispatch_loop 异步任务。
- 步骤 3 启动客户端：client.start() 启动传输层并创建 _read_loop 异步任务。
- 步骤 4 客户端发请求：client.call_tool('calculator', {'expression': '1+2'}) -> 生成 UUID -> 构造 MCPMessage -> transport.send() -> 等待 Future。
- 步骤 5 服务器收消息：_dispatch_loop 从 transport.receive() 获取消息 -> _process_message() 查找 handler -> _handle_tools_call() 调用 registry.execute_tool()。
- 步骤 6 工具执行：registry.execute_tool() -> tool.safe_execute() -> validate_params() + execute() -> 返回 ToolResult (含 success/output/error/execution_time)。
- 步骤 7 服务器回响应：构造 MCPMessage.create_response(request_id, result) -> transport.send()。
- 步骤 8 客户端收响应：_read_loop 读取消息 -> 匹配 _pending[id] -> Future.set_result(result) -> 客户端拿到结果。

2.8 内置工具 (6 个)

工具名	分类	描述
echo	system	回显输入内容，支持大写转换，用于测试协议链路
calculator	data	数学表达式计算，使用 AST 安全解析，防止代码注入
text_processor	data	文本统计与变换：字符数/词数/行数、大小写/反转

time	system	获取当前时间，支持 iso/timestamp/datetime/date 格式和时区偏移
json_formatter	code	JSON 格式化/压缩/校验，支持自定义缩进
random_generator	data	随机数/随机字符串/UUID/choice 生成，支持种子复现

三、项目过程中遇到的问题

3.1 ToolNotFoundError 继承 KeyError 导致错误消息重复包装

问题描述：初始设计中 ToolNotFoundError 继承自 KeyError，在服务器的 `_process_message` 中通过 `except` 捕获后，KeyError 的字符串表示会额外添加引号和重复消息，导致客户端收到的错误消息为「工具未找到: '工具未找到: nonexistent_tool'」，出现双重包装。

解决方案：将 ToolNotFoundError 的父类从 KeyError 改为 Exception，并在服务器的异常处理中新增 `except ToolNotFoundError` 分支，使用 `str(e)` 获取干净的错误消息。同时新增 `except ToolError` 分支处理工具业务错误，返回 `TOOL_EXECUTION_ERROR` 错误码，与工具未找到 (`TOOL_NOT_FOUND`) 区分。

收获：理解了 Python 异常继承体系对错误处理的影响，KeyError 的 `__str__` 方法会对消息加引号，不适合作为业务异常的父类。在设计异常层次时，应优先继承 Exception 或自定义基类，避免标准库异常的隐式行为干扰业务逻辑。

3.2 Pydantic V2 字段名冲突：schema 字段遮蔽 BaseModel 方法

问题描述：ToolInfo 模型中定义了 `schema: ToolSchema` 字段，但 Pydantic V2 的 BaseModel 保留了 `schema()` 方法 (V1 遗留兼容)，运行时产生 `UserWarning: Field name "schema" shadows an attribute in parent "BaseModel"`。

解决方案：将字段名从 `schema` 重命名为 `tool_schema`，并更新所有引用 (`registry.py`、`server.py`、`run.py` 中的 `info.schema` / `t.schema` 改为 `info.tool_schema` / `t.tool_schema`)。消除警告的同时，字段名也更语义化，避免与 Pydantic 内置方法冲突。

收获：掌握了 Pydantic V2 的迁移要点，V2 中 `BaseModel.schema()` 已废弃 (改为 `model_json_schema()`)，但为兼容仍保留方法名，导致字段命名冲突。设计模型时应避免使用 `schema`、`json`、`dict` 等保留名。

3.3 InProcessTransport 抽象类实例化失败

问题描述：InProcessTransport 最初继承自 TransportBase (ABC)，但本身未实现 `start/stop/send/receive/is_connected` 五个抽象方法 (这些方法在内部类 `ServerSide` 和 `ClientSide` 中实现)，导致 `TypeError: Can't instantiate abstract class InProcessTransport with abstract methods...`

解决方案：重新审视设计语义——InProcessTransport 是一个「传输对」工厂，不是真正的传输层实现。将其父类从 TransportBase 改为普通类 (object)，通过 `server_side()` / `client_side()` 返回的 `ServerSide` / `ClientSide` 才继承 TransportBase。同时更新 `create_transport` 工厂函数的返回类型注解为 Any，并补充文档说明。

收获：理解了 ABC（抽象基类）的实例化约束 —— 子类必须实现所有抽象方法才能实例化。在设计工厂模式时，工厂类本身不应继承产品类的抽象基类，应通过组合而非继承关联。

3.4 工具状态管理：业务错误与系统错误的区分

问题描述：初始设计中，工具执行失败（包括除零、参数错误等业务错误）后，状态被设为 ERROR，后续所有调用都因「状态不可用」而被拒绝。一个除零错误就永久禁用了 calculator 工具，导致后续合法表达式也无法执行。

解决方案：区分业务错误和系统错误的处理策略。safe_execute 将所有异常包装为 success=False + error 消息，其中业务错误（ToolError）的 error 是原始消息，系统错误（未知异常）的 error 以「内部错误:」前缀标识。registry.execute_tool 根据前缀判断：业务错误保持 ACTIVE 状态（工具仍可用），只有系统错误才标记为 ERROR。同时细化状态检查：INACTIVE 和 ERROR 都不可执行，但错误消息不同。

收获：理解了工具状态管理的语义设计 —— 业务错误（如除零、参数错误）是工具正常工作的一部分，不应影响工具可用性；只有系统级故障（如内存损坏、状态不一致）才应标记为不可用。这与 HTTP 服务的 4xx（客户端错误）vs 5xx（服务器错误）的设计哲学一致。

3.5 Calculator 工具的安全计算：AST 解析防止代码注入

问题描述：Calculator 工具需要计算用户提供的数学表达式，直接使用 eval() 存在严重安全风险（如 import os; os.system('rm -rf /')），需要一种既安全又能支持基本数学运算的方案。

解决方案：使用 Python 的 ast 模块解析表达式为 AST（抽象语法树），然后通过白名单方式递归计算：只允许 Expression、Constant（数字）、Bin Op（加减乘除幂模）、UnaryOp（正负号）四种节点类型，其他节点（如 Call、Attribute、Import）直接拒绝。二元运算符通过字典映射到 operator 模块的函数，捕获 ZeroDivisionError 返回友好错误。

收获：掌握了 Python AST 解析的安全应用场景。相比 eval()，AST 方式通过白名单控制可执行的语法元素，是处理用户输入表达式的标准安全方案。类似思路可用于 JSON Path 查询、规则引擎等需要解析类代码输入的场景。

3.6 异步消息分发与请求-响应匹配

问题描述：MCP 客户端发送请求后需要等待服务器响应，但传输层是异步消息流（receive 返回 AsyncIterator），如何将响应消息正确匹配到对应的请求？多个并发请求如何同时等待？

解决方案：基于 request_id + asyncio.Future 的匹配机制。客户端每发一条请求生成唯一 UUID 作为 id，在 _pending 字典中存储 id -> Future 映射。后台 _read_loop 持续读取消息流，收到响应/错误消息时根据 id 从 _pending 取出 Future，调用 set_result 或 set_exception 完成异步等待。多个并发请求各自有独立的 Future，互不阻塞。超时通过 asyncio.wait_for 实现，超时后从 _pending 移除避免内存泄漏。

收获：理解了异步 RPC 的核心设计模式 —— Future/Promise 机制。这与 gRPC、WebSocket
RPC 等框架的底层原理一致：通过唯一 ID
关联请求和响应，利用事件循环的并发能力支持多路复用。是分布式通信的基础模式。

3.7 服务器消息分发与异常分层处理

问题描述：服务器收到消息后需要分发到对应的处理器，但处理器可能抛出多种异常（参数错误、工具未找到、工具执行错误、内部错误），如何统一处理并返回合适的错误码？

解决方案：在 `_process_message` 中实现分层异常处理，按异常类型匹配不同的错误码：(1)
`ValueError` -> `INVALID_PARAMS` (-32602) ; (2) `ToolNotFoundError` ->
`TOOL_NOT_FOUND` (-32001) ; (3) `ToolError` ->
`TOOL_EXECUTION_ERROR` (-32002, 业务错误) ; (4) 其他异常 ->
`INTERNAL_ERROR` (-32603, 系统错误, 记录完整堆栈)。通知消息（无
id）单独处理，不返回响应。每条消息通过 `asyncio.create_task` 异步处理，不阻塞接收循环。

四、面试官可能提出的问题

4.1 架构设计类

Q1: 为什么选择 JSON-RPC 2.0 作为 MCP 协议的基础?

JSON-RPC 2.0 是成熟的远程调用标准，选择它有三个原因：(1) 语义完整 —— 天然支持请求/响应/通知/错误四种消息类型，覆盖工具调用的所有场景；(2) 简单易解析 —— JSON 格式，任何语言都能轻松实现客户端和服务端；(3) 生态广泛 —— VS Code、Language Server Protocol 等都基于 JSON-RPC，开发者熟悉度高。相比 gRPC (需 protobuf)、GraphQL (过于复杂)，JSON-RPC 在工具调用场景下是复杂度和功能性的最佳平衡点。

Q2: 传输层为什么要支持三种方式？不能统一用一种吗？

因为不同场景对传输的要求不同：(1) InProcess 适用于测试和单进程场景，无网络开销，调试方便；(2) SSE 适用于服务器推送场景（如工具执行进度），基于 HTTP 兼容性好，穿透防火墙容易；(3) WebSocket 适用于频繁双向交互场景，全双工通信效率最高。通过 TransportBase 抽象基类统一接口，上层服务器/客户端代码不感知传输差异，实现「传输无关」的协议层。这是策略模式的典型应用。

Q3: InProcessTransport 为什么设计成「传输对」工厂而不是直接实现 TransportBase?

因为进程内通信天然就是双向的 —— 服务器和客户端在同一个进程，共享队列。如果 InProcessTransport 直接实现 TransportBase，它只能代表一端（服务器或客户端），另一端需要额外的引用来访问共享队列，耦合度高。设计成工厂后，InProcessTransport 持有两条共享队列，通过 server_side() / client_side() 返回两个独立的 TransportBase 实现，各自只暴露自己视角的 send/receive，封装更清晰。这也是它不能继承 TransportBase 的原因 —— 它不是传输层，是传输层的工厂。

Q4: ToolRegistry 为什么用组合而不是继承来管理工具？

因为注册中心与工具是「拥有」关系而非「是」关系。一个 Registry 拥有多个工具，工具可以动态增减，工具的实现不依赖 Registry。如果用继承 (Registry is-a Tool)，语义错误且只能管理一个工具。组合方式更灵活：工具独立实现 execute 逻辑，Registry 只负责注册/查找/执行/统计，职责单一。这也是「组合优于继承」设计原则的体现。

4.2 技术细节类

Q5: 客户端如何将响应消息匹配到对应的请求？

基于 request_id + asyncio.Future 的匹配机制。客户端发送请求时生成唯一 UUID 作为消息 id，在 _pending 字典中存储 id -> Future 映射，然后 await Future 等待结果。后台 _read_loop 持续读取消息流，收到响应消息时根据 id 从 _pending 取出 Future，调用 set_result

完成等待。这支持多个请求并发等待（每个有独立 Future），是异步 RPC 的标准模式。超时通过 `asyncio.wait_for` 控制，超时后从 `_pending` 移除避免泄漏。

Q6: Calculator 工具如何防止恶意代码注入？

使用 AST（抽象语法树）白名单解析，而非 `eval()`。通过 `ast.parse(expression, mode='eval')` 将表达式解析为 AST，然后递归遍历 AST 节点，只允许四种类型：Expression（根节点）、Constant（数字字面量）、BinOp（二元运算：加减乘除幂模）、UnaryOp（一元运算：正负号）。其他节点（如 Call 函数调用、Attribute 属性访问、Import 导入）直接拒绝并抛出 `ToolError`。二元运算符通过字典映射到 `operator` 模块的函数，避免直接使用 `eval`。这是处理用户输入表达式的标准安全方案。

Q7: 工具状态管理中，业务错误和系统错误如何区分？

通过 `safe_execute` 的 `error` 消息前缀区分。`safe_execute` 捕获所有异常：`ToolError`（业务错误，如除零、参数校验失败）的 `error` 是原始消息；未知异常（系统错误，如 `AttributeError`、`TypeError`）的 `error` 以「内部错误:」前缀标识。`registry.execute_tool` 根据前缀判断：业务错误保持 `ACTIVE` 状态（工具仍可用），系统错误标记为 `ERROR`（工具不可用）。这保证了 `calculator` 处理 I/O 后仍能正常计算 `1+1`，只有工具本身代码有 bug 才会停用。

Q8: 为什么用 asyncio 而不是多线程？

因为 MCP 框架的工作负载是 I/O 密集型——消息收发、工具调用（可能涉及网络请求）、SSE/WebSocket 通信都是 I/O 操作。`asyncio` 的协程模型在 I/O 等待时释放事件循环处理其他任务，单线程避免了多线程的锁竞争和上下文切换开销。且 `asyncio.Future` 与 `asyncio.Queue` 天然集成，请求-响应匹配和消息流处理非常简洁。如果工具涉及 CPU 密集型任务（如大量计算），可以在工具内部用 `run_in_executor` 委托给线程池。

Q9: Config 单例模式为什么要用双重检查锁（DCL）？

因为框架可能在多线程环境中被调用（如 Web 服务嵌入 MCP 服务器），单例初始化若不加锁会导致创建多个实例；若整个方法加锁则性能差。DCL 先检查实例是否存在，不存在再加锁，加锁后再次检查（防止等待期间其他线程已创建），兼顾线程安全与性能。Python 中由于 GIL 的存在，简单的模块级单例也能工作，但 DCL 是更通用、更严谨的做法，在面试中体现对并发的理解。

4.3 工程实践类

Q10: 如何扩展一个新的自定义工具？

三步即可：(1) 继承 `ToolBase`，定义 `name`、`description`、`category`、`parameters_schema` 等类属性；(2) 实现 `async execute(params)` 方法，返回 JSON 可序列化结果；(3) 调用 `registry.register(MyTool())`

注册到注册中心。框架自动处理参数校验、异常捕获、调用统计、Schema 暴露。如果需要远程注册（只暴露元信息不提供实现），可通过 `client.register_tool(dict)` 实现。

Q11: MCP 协议解决了什么问题？相比直接用 Function Calling 有什么优势？

MCP 解决「工具碎片化」问题。Function Calling 是 LLM 厂商各自的实现，OpenAI、Anthropic、Google 的接口不兼容，工具代码无法跨模型复用。MCP 统一了工具的注册（`tools/register`）、发现（`tools/list`）、调用（`tools/call`）格式，任何 MCP 兼容的工具服务器都能被任何 MCP 客户端使用，解耦了工具实现与 LLM 调用。另一个优势是支持远程工具——工具不必和 Agent 在同一进程，通过 SSE/WebSocket 远程调用，适合微服务架构和工具共享场景。

Q12: 服务器如何处理并发请求？消息处理会阻塞接收吗？

不会阻塞。服务器的 `_dispatch_loop` 从 `transport.receive()` 获取消息后，通过 `asyncio.create_task(_process_message(message))` 异步处理每条消息，不等待处理完成就继续接收下一条。`_process_message` 内部调用 `handler` 也是异步的。这意味着多个请求可以并发执行，充分利用 I/O 等待时间。工具执行通过 `asyncio.wait_for` 设置超时，避免单个慢工具阻塞整个服务器。

Q13: 项目有哪些可改进的方向？

(1) 认证授权：当前缺乏身份验证，生产环境需增加 API Key / JWT 认证；(2) 工具版本管理：支持同一工具多版本共存，按版本号路由调用；(3) 工具依赖管理：工具 A 可调工具 B，支持工具链编排；(4) 流式返回：长时间工具支持流式返回进度（如文件下载）；(5) 多服务器联邦：客户端同时连接多个 MCP 服务器，统一工具发现；(6) 工具市场：基于 `tools/register` 实现插件化工具市场，支持动态安装；(7) 可观测性：增加 `trace` 追踪每条消息的完整链路，便于调试。

4.4 扩展问题

Q14: 如果要支持多服务器联邦，你会怎么设计？

基于现有 `MCPClient` 扩展：(1) 定义 `MultiServerClient` 管理多个 `MCPClient` 实例，维护 `server_id -> client` 映射；(2) `list_tools` 时聚合所有服务器的工具列表，工具名加 `server_id` 前缀避免冲突（如 `server1/calculator`）；(3) `call_tool` 时根据前缀路由到对应服务器；(4) 增加健康检查机制，定期 `ping` 各服务器，自动剔除不可用节点；(5) 支持工具偏好配置，同一工具名多服务器提供时按优先级选择。关键挑战是工具命名空间和故障转移。

Q15: 这个项目最难的地方是什么？

不是单一技术点，而是「协议工程的完整性」：(1) 异步消息分发与请求-响应匹配的并发控制，要保证 `Future` 正确匹配、超时正确清理、异常正确传播；(2) 传输层抽象的边界——`InProcessTransport`

是工厂还是传输？这个设计决策影响了整个类的继承体系；(3) 工具状态管理的语义设计 —— 业务错误不该停用工具，但要区分业务错误和系统错误需要精巧的设计；(4) Calculator 的安全计算 —— 既要支持数学表达式又要防止注入，AST 白名单是正确但需要仔细设计的方案；(5) 异常分层处理 —— 服务器要区分参数错误、工具未找到、工具执行错误、内部错误，每种对应不同的错误码和处理策略。

附录：项目技术栈速查

类别	技术 / 库	用途
语言	Python 3.11	主开发语言，异步生态成熟
数据模型	Pydantic V2	Schema 定义、数据校验、序列化
协议	JSON-RPC 2.0	MCP 消息格式基础
HTTP 框架	aiohttp	SSE / WebSocket 传输层实现
异步	asyncio	协程并发、Future、Queue
安全计算	ast	Calculator 工具的 AST 安全解析
配置	tomllib	TOML 配置文件解析
日志	logging	结构化日志、多级别输出
传输协议	SSE / WebSocket	远程通信（可选）

提示：面试时重点突出「为什么这样设计」而非「实现了什么」，展现工程权衡能力。对于 MCP 协议的理解要深入到 JSON-RPC 消息格式层面，对于异步架构要能讲清楚 Future 匹配机制，对于工具管理要区分业务错误和系统错误的不同处理策略。